

# PATHIQ AI

*Awareness Before Impact.*

---

## Intern Implementation & Execution Handbook

Phase 1 MVP — Edge-First Road Hazard Detection System

**Anviva Technologies Pvt. Ltd.**

Honda Bharat Safety Program | AIC IIT Delhi

Version: v1.0

Date: 27 July 2026

Classification: Internal — Engineering Interns & Mentors

Contact: [anviva.pvt.ltd@gmail.com](mailto:anviva.pvt.ltd@gmail.com) | +91 88658 59049

## Table of Contents

---

1. Cover
2. Purpose of This Handbook
3. Product Overview
4. Team Roles & Responsibilities
5. Development Environment Setup
6. Repository Structure (Mandatory)
7. Git Workflow & PR Rules
8. Phase 1 Milestones (45-Day Internship)
9. AI/ML Workstream
10. Database & Sync Contract
11. API Contract (FastAPI)
12. Performance Targets (Non-Negotiable)
13. Testing Plan
14. Security & Privacy Checklist
15. Deliverables Checklist
16. Handover Requirements
17. Appendices

## 2. Purpose of This Handbook

---

This handbook exists to convert the finalized PathIQ AI Technical Architecture Document (July 2026) into a day-to-day execution plan that an engineering intern can pick up and act on without needing further clarification from the founder or mentor. It translates architecture decisions into concrete tasks, file structures, schemas, contracts, and acceptance tests.

It is intended for four audiences:

- Flutter Intern — responsible for the mobile application: camera pipeline, TFLite integration, UI, local storage, and sync.
- AI/ML Intern — responsible for dataset preparation, model training, evaluation, quantization, and export of the on-device detection model.
- Technical Mentor — responsible for code review, unblocking interns, and verifying acceptance criteria at each milestone.
- Director / Product Owner (Founder, Anviva Technologies) — uses this handbook to track progress against the 45-day Phase 1 plan and to report status to the Honda Bharat Safety Program.

### Authoritative Reference

- The attached "PathIQ AI — Technical Architecture Document (July 2026)" remains the single source of truth for all technology and architecture decisions.
- This handbook does not override, replace, or re-litigate any decision in that document — it operationalizes it.
- If any instruction in this handbook appears to conflict with the architecture document, the architecture document takes precedence and the Technical Mentor should be notified.

### 3. Product Overview

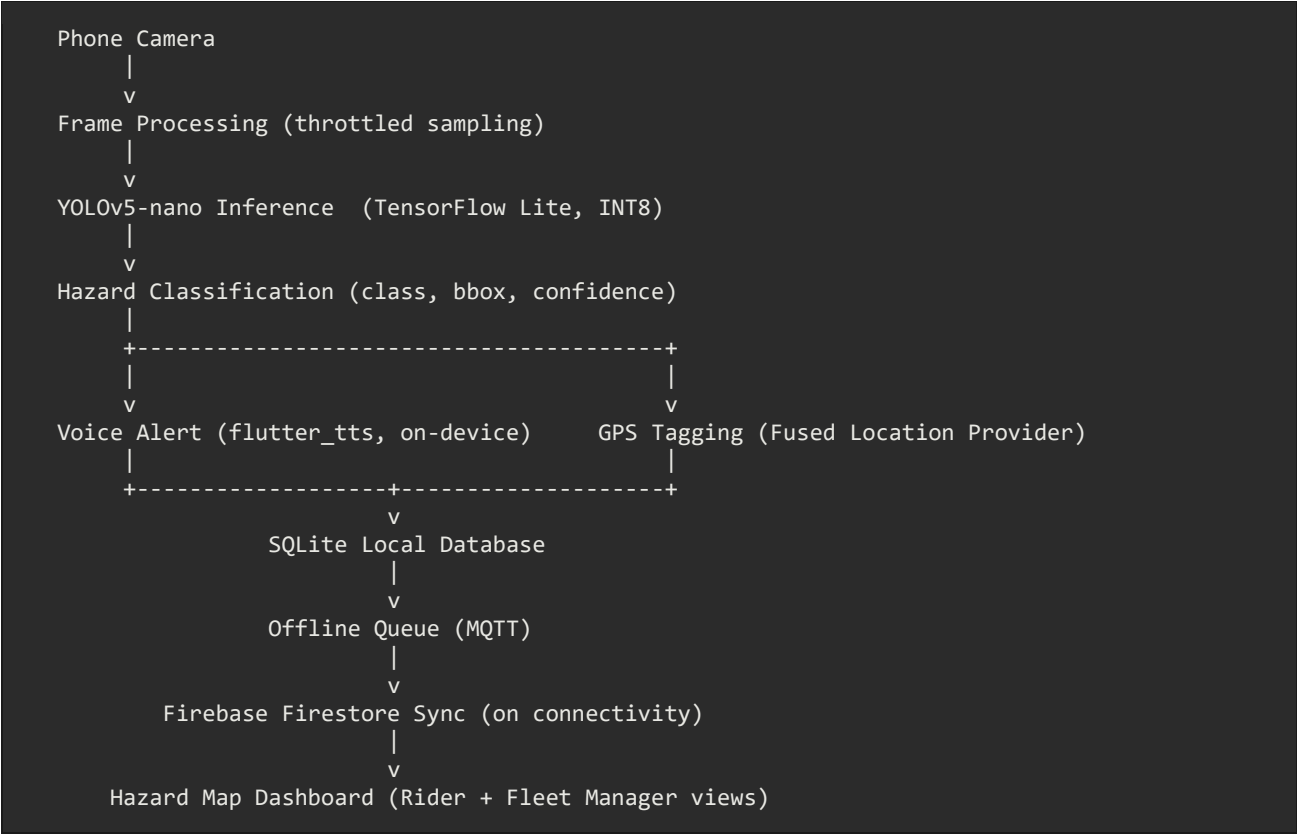
PathIQ AI turns a smartphone mounted on a two-wheeler into a real-time road-hazard co-pilot. The phone camera continuously observes the road ahead; an on-device AI model detects potholes, cracks, bumps, and waterlogging; the rider receives an immediate spoken alert; and every detection is GPS-tagged and logged so that a crowdsourced hazard map can be built for other riders and fleet managers.

Core Phase 1 capabilities:

- Real-time pothole and road-hazard detection using on-device inference (no cloud round-trip in the detection path).
- Offline, sub-100ms voice alerts via flutter\_tts — fully functional with zero network connectivity.
- GPS tagging of every hazard event using the Fused Location Provider (geolocator).
- Offline-first local storage (SQLite) guaranteeing zero data loss during a fully offline ride.
- MQTT-backed offline sync queue that uploads hazard events to Firebase Firestore once connectivity is restored.
- A community hazard map aggregating anonymized detections for riders and fleet managers.

Rider interaction is fully automatic — there is no manual hazard entry in Phase 1. The system detects, alerts, logs, and syncs without requiring the rider to touch the screen while riding.

#### 3.1 Data Flow (Edge-First)



This flow matches Section 5 of the Technical Architecture Document exactly. No step in this diagram may be reordered, removed, or routed through a server without explicit sign-off from the Director, because the edge-first guarantee (detection never depends on network signal) is the product's core safety and cost claim.

## 4. Team Roles & Responsibilities

R = Responsible (does the work), A = Accountable (final sign-off), C = Consulted, I = Informed.

| Workstream                                   | Flutter Intern | AI/ML Intern | Technical Mentor | Director |
|----------------------------------------------|----------------|--------------|------------------|----------|
| Repository setup & CI/CD                     | R              | C            | A                | I        |
| Camera frame pipeline                        | R              | I            | A                | I        |
| TFLite interpreter integration (mobile side) | R              | C            | A                | I        |
| Model training & evaluation                  | I              | R            | A                | I        |
| INT8 quantization & export                   | C              | R            | A                | I        |
| Mobile integration contract (tensor I/O)     | C              | C            | A/R              | I        |
| SQLite schema & local storage                | R              | I            | A                | I        |
| MQTT offline sync                            | R              | I            | A                | I        |
| FastAPI endpoints                            | C              | C            | A/R              | I        |
| Dataset labeling & QC                        | I              | R            | A                | I        |
| Voice alert & GPS integration                | R              | I            | A                | I        |
| Hazard map UI                                | R              | I            | A                | I        |
| Release testing & benchmarking               | R              | R            | A                | I        |

Notes on ownership:

- Technical Mentor holds final Accountability (A) on all technical workstreams except FastAPI endpoints and the mobile integration contract, which are jointly owned (A/R) with the Technical Mentor because they define the boundary between the mobile app and the ML pipeline.
- Director is Informed (I) on all technical decisions and is Accountable only for scope, timeline, and Honda reporting — not for line-level technical implementation.
- Any change to an A/R boundary (e.g., who owns the tensor input/output contract) must be agreed in writing between the Flutter Intern and AI/ML Intern, with the Technical Mentor as tiebreaker.

## 5. Development Environment Setup

| Tool                                     | Minimum Version                                        | Verification Command                  |
|------------------------------------------|--------------------------------------------------------|---------------------------------------|
| Flutter SDK                              | 3.22.x (stable channel) or newer                       | flutter --version                     |
| Dart SDK                                 | Bundled with Flutter 3.22.x (Dart 3.4+)                | dart --version                        |
| Android Studio                           | Iguana (2023.2.1) or newer, with Android SDK 34        | studio.sh --version (or Help > About) |
| Python                                   | 3.10 or 3.11                                           | python3 --version                     |
| CUDA (optional, for local training only) | CUDA 11.8 / cuDNN 8.6, only if training on a local GPU | nvidia-smi                            |
| Git                                      | 2.40 or newer                                          | git --version                         |
| Firebase CLI                             | 13.x or newer                                          | firebase --version                    |
| Fastlane                                 | 2.222.x or newer (Ruby 3.x)                            | fastlane --version                    |

### 5.1 Setup Sequence

1. Install Git, Python 3.10/3.11, and Android Studio (with Android SDK 34 and an emulator image, e.g. Pixel 7 API 34).
2. Install Flutter via the official SDK archive or version manager (fvm recommended for team version consistency).
3. Run flutter doctor and resolve every reported issue before writing any code.
4. Install the Firebase CLI (npm install -g firebase-tools) and run firebase login.
5. Install Fastlane (gem install fastlane) — required only by the Flutter Intern for release automation, from Milestone M7 onward.
6. Clone the monorepo and follow Section 6 to confirm the expected folder structure exists locally.

### 5.2 Expected Output — flutter doctor

```
$ flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.22.x)
[✓] Android toolchain - develop for Android devices
[✓] Chrome - develop for the web
[✓] Android Studio (version 2023.2)
[✓] Connected device (1 available)
[✓] Network resources

• No issues found!
```

If any check fails (in particular the Android toolchain or a connected device), do not proceed to Milestone M1 — resolve it with the Technical Mentor first.

## 6. Repository Structure (Mandatory)

The project is a single monorepo. No workstream may create a second top-level repository without Director approval.

|                    |                                                         |
|--------------------|---------------------------------------------------------|
| /pathiq-ai         |                                                         |
| /mobile_app        | Flutter application (Flutter Intern owns)               |
| /lib               |                                                         |
| /core              | DI (get_it), routing (go_router), constants             |
| /features          |                                                         |
| /camera            | Camera capture + frame throttling                       |
| /inference         | TFLite interpreter service                              |
| /alerts            | flutter_tts voice alert service                         |
| /location          | geolocator GPS service                                  |
| /storage           | sqflite schema + repositories                           |
| /sync              | MQTT offline queue + Firestore sync                     |
| /hazard_map        | Map view (rider + fleet manager)                        |
| /models            | Bundled .tflite model + labels.txt                      |
| /test              | unit + widget tests                                     |
| /integration_test  | end-to-end/on-device tests                              |
| /backend           | FastAPI application (Flutter/Mentor owns)               |
| /app               |                                                         |
| /routers           | hazard, sync, fleet endpoints                           |
| /schemas           | Pydantic request/response models                        |
| /services          | aggregation, dedup logic                                |
| /tests             |                                                         |
| /ml_pipeline       | AI/ML Intern owns end-to-end                            |
| /datasets          | raw + processed images (not committed – see .gitignore) |
| /labels            | YOLO-format .txt annotations                            |
| /training          | train.py, hyperparameter configs                        |
| /evaluation        | eval.py, metrics reports                                |
| /export            | ONNX / SavedModel / .tflite export scripts              |
| /docs              | This handbook, architecture doc, ADRs                   |
| /.github/workflows | CI: lint, test, build, beta distribution                |

### 6.1 Directory Purpose Summary

| Directory          | Purpose                                                   | Primary Owner           |
|--------------------|-----------------------------------------------------------|-------------------------|
| /mobile_app        | Flutter app: camera, inference, alerts, storage, sync, UI | Flutter Intern          |
| /backend           | FastAPI sync & aggregation service                        | Flutter Intern + Mentor |
| /ml_pipeline       | Dataset, training, evaluation, export                     | AI/ML Intern            |
| /docs              | Handbook, architecture doc, decision records              | Technical Mentor        |
| /.github/workflows | CI/CD pipelines                                           | Technical Mentor        |



## 7. Git Workflow & PR Rules

### 7.1 Branch Naming

```
<type>/<milestone>-<short-description>
```

Examples:

```
feature/m2-camera-frame-throttling
feature/m3-tflite-interpreter-service
fix/m5-mqtt-retry-backoff
chore/m1-repo-scaffolding
```

### 7.2 Commit Message Format (Conventional Commits)

```
<type>(<scope>): <short summary>
```

```
<optional body explaining why, not just what>
```

Examples:

```
feat(camera): add frame throttling at 10 FPS cap
fix(sync): correct MQTT reconnect backoff timing
docs(ml): add dataset labeling QC checklist
```

### 7.3 Pull Request Template

```
## Summary
What does this PR do and which milestone/task does it close?

## Changes
-

## Testing Performed
- [ ] Unit tests pass locally
- [ ] Manually tested on a physical Android device
- [ ] Screenshots / logs attached below

## Acceptance Criteria Checklist
- [ ] Matches milestone acceptance criteria in Section 8

## Linked Issue
Closes #
```

### 7.4 Required Reviewers & Definition of Done

- Every PR requires at least one review from the Technical Mentor before merge; ML-pipeline PRs additionally require a peer review from the Flutter Intern only when they touch the mobile integration contract (Section 9.7).
- No PR merges to main with a failing CI check (lint, unit tests, build).
- Definition of Done = code merged + tests passing + acceptance criteria for the corresponding milestone task verified + documentation (docstrings/README) updated.

## 8. Phase 1 Milestones (45-Day Internship)

The 45-day plan below sequences work without fixed calendar dates so it can start on any onboarding day; durations are elapsed working days per milestone, not clock time.

| ID | Days  | Owner                                  | Focus                                               |
|----|-------|----------------------------------------|-----------------------------------------------------|
| M1 | 1–5   | Flutter Intern + Mentor                | Project setup, Firebase config, app skeleton        |
| M2 | 6–12  | Flutter Intern                         | Camera frame pipeline, background service           |
| M3 | 13–20 | Flutter Intern + AI/ML Intern          | TFLite integration, inference service, overlay      |
| M4 | 21–27 | Flutter Intern                         | Voice alerts, GPS tagging, SQLite, ride logging     |
| M5 | 28–34 | Flutter Intern                         | MQTT offline queue, Firestore sync, dedup           |
| M6 | 35–40 | Flutter Intern                         | Hazard map, ride summary, analytics/crash reporting |
| M7 | 41–45 | Flutter Intern + AI/ML Intern + Mentor | E2E testing, benchmarking, release candidate        |

### Milestone M1 (Days 1–5)

**Owner: Flutter Intern (with Technical Mentor)**

Tasks:

- Set up the monorepo structure exactly as defined in Section 6.
- Create the Firebase project; configure Firebase Auth (phone-OTP), Firestore, and Crashlytics.
- Scaffold the Flutter app skeleton with Riverpod, get\_it, and go\_router wired up but empty.
- Configure GitHub Actions for lint + test on every PR.

Deliverables:

- Working monorepo pushed to GitHub with the folder structure from Section 6.
- Firebase project connected to the Flutter app (firebase\_options.dart generated).
- Empty but running app skeleton installable on a physical Android device.

#### Acceptance Criteria

- flutter doctor shows zero issues on the intern's machine.
- App skeleton builds and launches on a physical Android device with no crash.
- CI pipeline runs lint + test on a sample PR and passes.
- Firebase Auth phone-OTP sign-in succeeds end-to-end in a test build.

## Milestone M2 (Days 6–12)

**Owner: Flutter Intern**

Tasks:

- Integrate the camera plugin for real-time frame streaming.
- Implement frame throttling (sampling at a fixed rate, not every camera frame) to control CPU/battery load.
- Implement a foreground/background service so detection continues while the app is minimized during a ride.

Deliverables:

- Camera preview screen streaming frames at a configurable throttled rate.
- Background service that keeps capturing while the app is minimized.

### Acceptance Criteria

- Frame throttling is configurable and defaults to a rate consistent with the  $\geq 10$  FPS inference target in Section 12.
- App continues capturing frames for at least 30 minutes with the screen off / app backgrounded, verified on a physical device.
- No camera-related memory leak after a 30-minute continuous run (verified via Android Profiler).

## Milestone M3 (Days 13–20)

**Owner: Flutter Intern (mobile integration) + AI/ML Intern (model handoff)**

Tasks:

- Integrate the TFLite interpreter into the Flutter app per the contract in Section 9.7.
- Load the exported YOLOv5-nano INT8 .tflite model and labels file bundled with the app.
- Build the inference service that consumes camera frames and returns hazard class + bounding box + confidence.
- Render a bounding-box overlay on the camera preview for debugging (developer-mode only; not shown to end riders in production builds).

Deliverables:

- Working inference service producing detections on-device.
- Debug bounding-box overlay toggle.
- First integrated .tflite model from the AI/ML Intern loaded successfully.

### Acceptance Criteria

- End-to-end inference latency  $\leq 90$  ms on a Snapdragon 7-series (or equivalent mid-range) test device.
- Sustained inference throughput  $\geq 10$  FPS during a 10-minute test ride recording.
- Model loads without crashing across at least 2 different physical Android devices.

## Milestone M4 (Days 21–27)

**Owner: Flutter Intern**

**Tasks:**

- Integrate flutter\_tts for offline voice alerts triggered above the confidence threshold defined in Section 9.7.
- Integrate geolocator (Fused Location Provider) to timestamp-match each detection to a GPS coordinate.
- Implement the SQLite schema from Section 10 for ride\_sessions and hazard\_events.
- Implement ride-session lifecycle logging (start ride, log events, end ride).

**Deliverables:**

- Voice alert fires within the same detection cycle as a qualifying hazard.
- Every hazard event stored in SQLite includes accurate lat/long and timestamp.
- Ride session start/stop UI flow.

**Acceptance Criteria**

- Voice alert latency from detection to spoken output is  $\leq 100$  ms with no network dependency (airplane mode test passes).
- 100% of hazard events written during a test ride have a valid, non-null GPS coordinate captured within 2 seconds of detection.
- Zero data loss verified: a full 2-hour ride with the device in airplane mode produces the same event count in SQLite as visual inspection of a reference recording.

**Milestone M5 (Days 28–34)****Owner: Flutter Intern****Tasks:**

- Implement the MQTT-backed offline sync queue per Section 10.
- Implement Firestore sync on connectivity restoration, including retry with backoff.
- Implement a deduplication contract so repeated sync attempts never create duplicate hazard records.

**Deliverables:**

- Working offline queue that persists across app restarts.
- Sync success reporting visible in a debug screen (queued / synced / failed counts).

**Acceptance Criteria**

- $\geq 99\%$  of queued events sync successfully within 5 minutes of connectivity being restored, tested across at least 3 connectivity-restoration scenarios.
- No duplicate hazard\_events documents appear in Firestore after forced repeated sync retries (verified by document-ID dedup key).
- App recovers a non-empty offline queue correctly after a full app kill and restart.

**Milestone M6 (Days 35–40)****Owner: Flutter Intern****Tasks:**

- Build the hazard map view rendering synced + local hazard points for riders and fleet managers.

- Build the ride-summary screen (distance, duration, hazards encountered, per-class counts).
- Wire up Firebase Analytics and Crashlytics events for key user actions and crashes.

Deliverables:

- Hazard map screen with map tiles and hazard markers.
- Ride summary screen shown at the end of each ride.
- Analytics events visible in the Firebase console within 24 hours of a test session.

#### Acceptance Criteria

- Hazard map correctly renders both locally-stored and Firestore-synced points without duplication.
- Ride summary numbers match the raw SQLite event log for the same ride within a controlled test.
- A deliberately forced crash appears in Crashlytics within the expected reporting window.

### Milestone M7 (Days 41–45)

**Owner: Flutter Intern + AI/ML Intern + Technical Mentor**

Tasks:

- Run full end-to-end testing across the Testing Plan in Section 13.
- Run performance benchmarking against every target in Section 12.
- Cut a release candidate build via Fastlane and distribute through Firebase App Distribution.
- Complete the documentation handover in Section 16.

Deliverables:

- Signed release-candidate APK/AAB.
- Benchmark report against Section 12 targets.
- Completed handover package per Section 16.

#### Acceptance Criteria

- All Section 12 performance targets met or explicitly documented as a known gap with mitigation plan.
- Full regression pass of the Testing Plan (Section 13) with evidence attached.
- Release candidate installs cleanly on at least 3 physical devices spanning low/mid/high Android tiers.

## 9. AI/ML Workstream

### 9.1 Dataset Specification

| Field                           | Specification                                                                                                   |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Hazard classes                  | pothole, crack, bump, waterlogging (4 classes for Phase 1 MVP)                                                  |
| Image resolution                | 1280×720 minimum at capture; resized to 640×640 for training, matching YOLOv5-nano's default input              |
| Annotation format               | YOLO format — one .txt per image: <class_id> <x_center> <y_center> <width> <height>, normalized 0–1             |
| Train / validation / test split | 70% / 20% / 10%, split by ride session (not by individual frame) to avoid leakage between near-duplicate frames |
| Minimum images per class        | 800 labeled instances per class minimum for the Phase 1 MVP model; 2,000+ preferred before beta                 |
| Naming convention               | <session_id>_<frame_index>.jpg with a matching <session_id>_<frame_index>.txt label file                        |

### 9.2 Data Collection Protocol

- Phone mounting angle: handlebar-mounted, camera axis parallel to the road surface, tilted 5–10° downward so the road 3–15 meters ahead is in frame.
- Time-of-day coverage: collect across morning, midday, evening, and low-light/dusk conditions — do not collect only in good daylight.
- Weather coverage: dry, overcast, and light-rain/wet-road conditions (waterlogging class requires wet-condition footage by definition).
- Urban vs highway coverage: both dense urban traffic roads and open highway stretches, since hazard appearance and background clutter differ significantly.
- Safety guidelines for riders: no data collection at speeds where reviewing the phone screen is required while riding; mounting must not obstruct the rider's view or vehicle controls; data collection rides follow all local traffic laws and use of the app must never distract the rider from primary riding attention.

### 9.3 Labeling Guidelines

- Bounding boxes must tightly enclose the visible hazard extent — no padding added for "safety margin."
- Occlusion handling: label a hazard if at least 40% of it is visible; otherwise skip the frame for that instance rather than guessing the boundary.
- Minimum object size: do not label hazards smaller than roughly 1% of frame area — these are not reliably detectable at the target inference resolution and introduce label noise.
- Ambiguous cases (e.g., shadow vs. crack, wet patch vs. waterlogging) are flagged in a review queue rather than guessed, and resolved by the AI/ML Intern with the Technical Mentor.
- Quality-control checklist: every labeled batch is spot-checked at a 10% sample rate for bounding-box tightness, correct class assignment, and missed instances before being merged into the training set.

### 9.4 Training Pipeline

```
# 1. Prepare dataset (from /ml_pipeline/datasets + /ml_pipeline/labels)
python ml_pipeline/training/prepare_dataset.py \
  --images ml_pipeline/datasets/raw \
  --labels ml_pipeline/labels \
  --out ml_pipeline/datasets/processed \
  --split 0.7 0.2 0.1

# 2. Train YOLOv5-nano
python train.py \
  --img 640 \
  --batch 32 \
  --epochs 150 \
  --data pathiq_hazards.yaml \
  --weights yolov5n.pt \
  --hyp hyp.scratch-low.yaml \
  --patience 20 \
  --project ml_pipeline/training/runs \
  --name pathiq_v1

# Key hyperparameters (hyp.scratch-low.yaml overrides)
lr0: 0.01
momentum: 0.937
weight_decay: 0.0005
mosaic: 1.0

# Early stopping: --patience 20 (stop if no mAP@0.5 improvement for 20 epochs)
# Checkpointing: best.pt and last.pt saved automatically per epoch under
runs/pathiq_v1/weights/
```

## 9.5 Evaluation

| Metric                                              | Minimum Acceptance Threshold (MVP)                                        |
|-----------------------------------------------------|---------------------------------------------------------------------------|
| mAP@0.5 (all classes)                               | ≥ 0.70                                                                    |
| Precision (all classes)                             | ≥ 0.70                                                                    |
| Recall (all classes)                                | ≥ 0.65                                                                    |
| F1 score (all classes)                              | ≥ 0.68                                                                    |
| Per-class mAP@0.5 (pothole — safety-critical class) | ≥ 0.75                                                                    |
| Confusion matrix                                    | Reviewed manually — no class confused >15% of the time with another class |

The evaluation script must output a confusion matrix, per-class precision/recall, and an overall metrics summary as a single evaluation report (see Appendix 17.4 template).

## 9.6 Quantization & Export

- Export the trained PyTorch checkpoint to ONNX (intermediate step, used internally for validation only, not shipped).
- Convert ONNX to a TensorFlow SavedModel.
- Generate an INT8 representative dataset: 200–500 real, diverse frames sampled from the validation set (not training set) covering all four hazard classes and lighting conditions.

10. Run TFLite conversion with INT8 post-training quantization using the representative dataset from the previous step.
11. Verify the exported .tflite model size is within the Section 12 target ( $\leq 2.5$  MB).
12. Run an accuracy regression check: re-evaluate mAP@0.5 on the INT8 model against the FP32 baseline; the drop must not exceed 3 percentage points absolute.

```
python ml_pipeline/export/export_tflite.py \
--weights runs/pathiq_v1/weights/best.pt \
--onnx-out ml_pipeline/export/pathiq_v1.onnx \
--saved-model-out ml_pipeline/export/pathiq_v1_saved_model \
--repr-dataset ml_pipeline/datasets/processed/val \
--repr-count 300 \
--tflite-out ml_pipeline/export/pathiq_v1_int8.tflite \
--quantize int8

python ml_pipeline/evaluation/eval_regression.py \
--fp32 runs/pathiq_v1/weights/best.pt \
--int8 ml_pipeline/export/pathiq_v1_int8.tflite \
--val-set ml_pipeline/datasets/processed/val
```

## 9.7 Mobile Integration Contract

| Parameter               | Specification                                                                                                                                               |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input tensor shape      | [1, 640, 640, 3], uint8 (INT8-quantized model)                                                                                                              |
| Normalization           | Pixel values 0–255 uint8, no float normalization — quantization parameters are embedded in the .tflite model's input tensor metadata                        |
| Output tensor format    | [1, N, 9] — N candidate detections, each: [x, y, w, h, objectness, class_0..class_3 scores], per YOLOv5 export convention                                   |
| Confidence threshold    | 0.45 default (configurable per class post-beta based on false-positive rate)                                                                                |
| NMS threshold (IoU)     | 0.45 default                                                                                                                                                |
| Model versioning scheme | pathiq_model_v<MAJOR>.<MINOR>.<PATCH>.tflite — MAJOR changes classes, MINOR changes training data materially, PATCH is re-training on the same data/classes |

This contract is the hard boundary between the AI/ML Intern and the Flutter Intern. Any change to input shape, output format, or thresholds must be a versioned model release communicated in the #pathiq-ml-mobile channel and reflected in the model's accompanying labels.txt and version string — not silently changed.



## 10. Database & Sync Contract

### 10.1 SQLite Schema

```
CREATE TABLE ride_sessions (
  session_id    TEXT PRIMARY KEY,
  user_id       TEXT NOT NULL,
  start_time    INTEGER NOT NULL,    -- epoch millis
  end_time      INTEGER,             -- null while ride in progress
  distance_meters REAL DEFAULT 0,
  sync_status   TEXT NOT NULL DEFAULT 'PENDING' -- PENDING | SYNCED
);

CREATE TABLE hazard_events (
  event_id      TEXT PRIMARY KEY,    -- UUID, doubles as Firestore dedup key
  session_id    TEXT NOT NULL REFERENCES ride_sessions(session_id),
  hazard_class  TEXT NOT NULL,       -- pothole | crack | bump | waterlogging
  confidence    REAL NOT NULL,
  latitude      REAL NOT NULL,
  longitude     REAL NOT NULL,
  timestamp     INTEGER NOT NULL,    -- epoch millis
  model_version TEXT NOT NULL,
  sync_status   TEXT NOT NULL DEFAULT 'PENDING' -- PENDING | QUEUED | SYNCED | FAILED
);

CREATE TABLE sync_queue (
  queue_id      TEXT PRIMARY KEY,
  event_id      TEXT NOT NULL REFERENCES hazard_events(event_id),
  attempt_count INTEGER NOT NULL DEFAULT 0,
  last_attempt  INTEGER,
  next_retry_at INTEGER,
  status        TEXT NOT NULL DEFAULT 'QUEUED' -- QUEUED | IN_FLIGHT | FAILED
);
```

### 10.2 Firestore Schema

| Collection            | Document ID                                              | Key Fields                                                                                          |
|-----------------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| ride_sessions         | {session_id}                                             | user_id, start_time, end_time, distance_meters (types map SQLite → Firestore 1:1)                   |
| hazard_events         | {event_id} (same UUID as SQLite — this IS the dedup key) | session_id, hazard_class, confidence, geo (GeoPoint), timestamp, model_version                      |
| hazard_map_aggregates | {geohash_cell}                                           | hazard_count, dominant_class, last_updated — written by a Cloud Function, never directly by the app |

### 10.3 MQTT Topics & Payload

```
Topic naming:
pathiq/{user_id}/hazard_events/publish
pathiq/{user_id}/hazard_events/ack
```

```
Payload JSON schema (published):
{
  "event_id": "uuid-string",
  "session_id": "uuid-string",
  "hazard_class": "pothole",
  "confidence": 0.87,
  "latitude": 28.6139,
  "longitude": 77.2090,
  "timestamp": 1753600000000,
  "model_version": "1.0.0"
}
```

Deduplication is enforced by using event\_id (a client-generated UUID) as the Firestore document ID. A retried publish that reaches Firestore twice results in the same document being overwritten with identical data, not a duplicate — this satisfies the  $\geq 99\%$  sync success target in Section 12 without a separate server-side dedup pass.

## 11. API Contract (FastAPI)

| Item           | Specification                                                                                                           |
|----------------|-------------------------------------------------------------------------------------------------------------------------|
| Base URL       | https://api.pathiq.internal/v1 (placeholder — replaced with the deployed Cloud Run / VM URL at release)                 |
| Authentication | Firebase ID token passed as Authorization: Bearer <token> on every request; verified server-side via Firebase Admin SDK |

### 11.1 Endpoints

| Method | Path                         | Purpose                                                        | Auth                          |
|--------|------------------------------|----------------------------------------------------------------|-------------------------------|
| POST   | /v1/hazards/sync             | Bulk-accept queued hazard events from a device sync batch      | Required                      |
| GET    | /v1/hazards/map              | Return aggregated, anonymized hazard points for a bounding box | Required                      |
| GET    | /v1/rides/{session_id}       | Return ride summary for a completed session                    | Required                      |
| GET    | /v1/fleet/{fleet_id}/summary | Return fleet-level hazard exposure summary                     | Required (fleet manager role) |
| GET    | /v1/health                   | Liveness/readiness check for CI/CD and monitoring              | None                          |

### 11.2 Request / Response Examples

POST /v1/hazards/sync — Request:

```
{
  "events": [
    {
      "event_id": "8f14e45f-...",
      "session_id": "3c9e2a10-...",
      "hazard_class": "pothole",
      "confidence": 0.87,
      "latitude": 28.6139,
      "longitude": 77.2090,
      "timestamp": 1753600000000,
      "model_version": "1.0.0"
    }
  ]
}
```

POST /v1/hazards/sync — Response (200 OK):

```
{
  "accepted": 1,
  "duplicates": 0,
}
```

```
"rejected": 0  
}
```

### 11.3 Error Codes

| Code | Meaning                                                                       | Client Handling                                                           |
|------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| 400  | Malformed payload (schema validation failure)                                 | Log locally; do not retry until payload is fixed                          |
| 401  | Missing or invalid Firebase ID token                                          | Refresh token and retry once                                              |
| 403  | Valid token but insufficient role (e.g., fleet endpoint from a rider account) | Do not retry; surface to UI only for fleet-manager screens                |
| 429  | Rate limited                                                                  | Retry with exponential backoff, honoring Retry-After header               |
| 500  | Server error                                                                  | Retry with exponential backoff via the existing MQTT/sync retry mechanism |

## 12. Performance Targets (Non-Negotiable)

These targets are derived from the Technical Architecture Document's edge-first latency and cost rationale (Section 5). They are Milestone M7 exit criteria, not aspirational goals.

| Target                                              | Value                                                                      |
|-----------------------------------------------------|----------------------------------------------------------------------------|
| End-to-end alert latency (detection → spoken alert) | ≤ 90 ms                                                                    |
| Inference throughput                                | ≥ 10 FPS on a Snapdragon 7-series (or equivalent mid-range) device         |
| Model size (INT8 .tflite)                           | ≤ 2.5 MB                                                                   |
| App cold start time                                 | ≤ 3 s                                                                      |
| Battery drain                                       | ≤ 12% per hour of continuous ride use                                      |
| Offline event durability                            | 0 data loss during a 2-hour fully offline ride                             |
| Sync success rate                                   | ≥ 99% of queued events synced within 5 minutes of connectivity restoration |

Any target missed at Milestone M7 must be documented with root cause and a mitigation plan in the handover report (Section 16) — it is not acceptable to silently ship below target without disclosure to the Technical Mentor and Director.

## 13. Testing Plan

---

### 13.1 Unit Tests

Scope: pure logic — SQLite repositories, MQTT queue state transitions, confidence-threshold filtering, GPS/timestamp matching. Pass/Fail: all tests green in CI; no test skipped without a documented reason.

### 13.2 Widget Tests

Scope: individual Flutter widgets (ride-summary card, hazard-map marker rendering, permission-request dialogs) in isolation. Evidence: CI test report artifact.

### 13.3 Integration Tests

Scope: camera → inference → alert → storage pipeline running together on an emulator or device via flutter integration\_test. Evidence: recorded test run log + pass/fail summary.

### 13.4 On-Road Validation Tests

Scope: real rides on real roads with known hazards, comparing app-logged detections against a human-reviewed reference video. Pass/Fail: recall and precision on the reference ride match or exceed the Section 9.5 thresholds. Evidence: reference video + generated hazard log side by side.

### 13.5 Offline/Online Transition Tests

Scope: forcing airplane mode on/off mid-ride at multiple points to verify queueing and sync-on-reconnect behavior. Pass/Fail: matches the sync success and durability targets in Section 12. Evidence: before/after event counts in SQLite vs. Firestore.

### 13.6 Regression Tests for New Model Versions

Scope: every new .tflite model version is re-run through Section 9.5 evaluation and the Section 9.6 accuracy-regression check before being bundled into a new app build. Pass/Fail: no metric regresses below its Section 9.5 threshold. Evidence: evaluation report attached to the model's version entry in /docs.

## 14. Security & Privacy Checklist

---

Mirrors Section 10 of the Technical Architecture Document.

- No raw camera frame is ever written to disk or transmitted off-device — frames are processed in-memory for inference only.
- No frame transmission of any kind occurs, in production or debug builds shipped to testers.
- TLS is enforced for all Firestore and FastAPI traffic; no endpoint accepts plaintext HTTP in any non-local environment.
- SQLite database is stored in the app's private, sandboxed storage — never in shared/public device storage.
- Every runtime permission request (camera, location) includes a clear in-app rationale shown before the OS permission dialog.
- Hazard coordinates synced to the community map are decoupled from individual rider identity before aggregation (Section 10.2, `hazard_map_aggregates`).
- Data collection and retention follow DPDP 2023 principles: data minimization, purpose limitation, and explicit user consent for location data.

### 14.1 Pre-Release Checklist

- Debug-mode bounding-box overlay (Section 8, M3) is disabled in the release build.
- No hardcoded API keys or Firebase service-account credentials are committed to the repository.
- Permission rationale copy has been reviewed and matches the DPDP-aligned consent language.
- A fresh device install shows the intended permission flow with no unexpected auto-grants.

## 15. Deliverables Checklist

---

### 15.1 Flutter Intern

- Monorepo /mobile\_app scaffolded and building on a physical device (M1).
- Camera pipeline with throttled frame streaming and background service (M2).
- TFLite interpreter integrated and meeting the M3 latency/FPS acceptance criteria (M3).
- Voice alerts, GPS tagging, SQLite schema, and ride-session logging implemented (M4).
- MQTT offline queue and Firestore sync with dedup implemented and tested (M5).
- Hazard map view, ride summary, analytics/crash reporting wired up (M6).
- Signed release-candidate build distributed via Firebase App Distribution (M7).

### 15.2 AI/ML Intern

- Dataset meeting the minimum image-per-class thresholds in Section 9.1, correctly split and labeled.
- Trained YOLOv5-nano model meeting all Section 9.5 evaluation thresholds.
- INT8-quantized .tflite export meeting the Section 12 model-size target with a documented accuracy-regression check.
- Mobile Integration Contract (Section 9.7) document finalized and shared with the Flutter Intern.

### 15.3 Technical Mentor

- All PRs reviewed against the Definition of Done in Section 7.4.
- All seven milestone acceptance-criteria sets verified and signed off in writing.
- Final benchmark report (Section 12) reviewed and any missed targets documented with mitigation.



## 16. Handover Requirements

---

The following must be submitted on the final day of the internship (end of Milestone M7):

- All feature branches merged to main via reviewed PRs — no open, unmerged feature branches.
- Signed release APK/AAB (Fastlane build artifact).
- Final trained model file (.tflite), version-tagged per the Section 9.7 scheme.
- Complete training logs (loss curves, per-epoch metrics) from the training run that produced the final model.
- Final evaluation report (Section 9.5 metrics + confusion matrix), using the template in Appendix 17.4.
- Dataset manifest: image counts per class, train/val/test split counts, and labeling QC pass rate.
- Environment reproduction file (requirements.txt / pubspec.lock snapshot, Flutter/Dart/Python versions used).
- A final demo video (3–5 minutes) showing: app launch, a live ride with at least one real detection and voice alert, offline-to-online sync, and the hazard map view.

## 17. Appendices

### 17.1 Sample .env Template

```
# /backend/.env.example — copy to .env and fill in real values, never commit .env
FIREBASE_PROJECT_ID=pathiq-ai-prod
FIREBASE_SERVICE_ACCOUNT_PATH=./secrets/firebase-service-account.json
MQTT_BROKER_URL=mqtts://broker.pathiq.internal:8883
MQTT_USERNAME=
MQTT_PASSWORD=
API_ENV=development # development | staging | production
LOG_LEVEL=INFO
```

### 17.2 firebase\_options.dart Note

firebase\_options.dart is auto-generated by running flutterfire configure from the /mobile\_app directory. It must never be hand-edited, and the file is safe to commit (it contains public client config, not secrets) — but any Admin SDK service-account JSON used by the backend must never be committed and must live only in /backend/secrets/, which is .gitignore'd.

### 17.3 Sample MQTT Payload

```
{
  "event_id": "e7b8b7c0-4b3a-4f8e-9a1a-2f6a9b6e2b41",
  "session_id": "3c9e2a10-8b2b-4a4f-9f2e-6f1a2b3c4d5e",
  "hazard_class": "waterlogging",
  "confidence": 0.78,
  "latitude": 28.7041,
  "longitude": 77.1025,
  "timestamp": 1753600123456,
  "model_version": "1.0.0"
}
```

### 17.4 Sample Evaluation Report Template

```
PathIQ AI — Model Evaluation Report
Model version: pathiq_model_v1.0.0.tflite
Date: <date>
Dataset: <val set name>, N images

Overall metrics:
  mAP@0.5:    0.xx
  Precision:  0.xx
  Recall:     0.xx
  F1:         0.xx

Per-class mAP@0.5:
  pothole:    0.xx
  crack:      0.xx
  bump:       0.xx
  waterlogging: 0.xx

INT8 vs FP32 regression: -0.0x mAP@0.5 (threshold: <= 0.03)
Confusion matrix: attached as confusion_matrix.png
```

## 17.5 Troubleshooting Guide — Common Flutter / TFLite Issues

| Symptom                                     | Likely Cause                                                                 | Fix                                                                                                                   |
|---------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Interpreter fails to load model             | Model file not bundled correctly in pubspec.yaml assets                      | Confirm the .tflite path is listed under flutter: assets: in pubspec.yaml and re-run flutter clean && flutter pub get |
| Inference latency far above 90ms            | Running on emulator instead of physical device, or INT8 delegate not enabled | Always benchmark on a physical mid-range device; verify the NNAPI/XNNPACK delegate is active                          |
| Camera preview is black                     | Camera permission not granted or camera plugin not initialized before use    | Check runtime permission flow; ensure CameraController.initialize() completes before starting the preview             |
| GPS coordinates are null or (0,0)           | Location permission denied, or location services disabled at OS level        | Check both app-level permission and device-level location services toggle                                             |
| MQTT queue grows without syncing            | Broker unreachable or credentials misconfigured                              | Verify MQTT_BROKER_URL/credentials in .env and check broker connectivity separately with a CLI MQTT client            |
| Duplicate hazard events appear in Firestore | event_id not being reused as the Firestore document ID on retry              | Confirm the sync layer always writes with .doc(event_id).set(...), never .add(...)                                    |